

In the above example, *magazineName* will get stored into the separate space known as closure. So when you are returning the *name()*, the name function will access the closure for all the local variables of the parent function *info()*.

The closure looping problem

You should be very careful while dealing with loops within the closure. Consider the example shown below:

```
var funcs = {};
for (var i = 0; i < 3; i++) {           // let's create 3
  funcs[i] = function() {             // and store them in
    console.log("My value: " + i);     // each should log its
  };                                   value.
}
for (var j = 0; j < 3; j++) {           // and now let's run
  funcs[j]()                           each one to see
}
```

On execution of the above code, you will see the following output:

```
My value: 3
My value: 3
My value: 3
...instead of the expected...
My value: 0
My value: 1
My value: 2
```

This happens because, after the execution of the parent's function, the value of 'i' will be 3 in the closure space. So when the child function looks for the value of 'i' it will get 3 instead of the expected 0,1,2.



Note: JavaScript's scopes are function-level, not block-level, and creating a closure just means that the enclosing scope gets added to the lexical environment of the enclosed function.

Solution: You can correct the closure looping problem by creating a separate closure for every value of 'i', as follows:

```
var funcs = [];
for (var i = 0; i < 3; i++) {
  funcs[i] = (function(index) {
    return function() {
      console.log("My value: " + index);
    }
  })(i);
}
```

```
}
for (var j = 0; j < 3; j++) {
  funcs[j]()
}
```

In the above code, for every iteration of 'i' a new closure will be created. So now if *funcs[0]* tries to access the value of 'i' it will get zero. The same holds for the *funcs[1]* and *funcs[2]*, and hence you will get your expected output.

Objects

Everything in JavaScript is an object. In fact, arrays, dates, strings and even numbers are considered to be objects. JavaScript is a prototype-based language and all the OOP concepts are directly implemented by the object itself.

Ways of creating an object

//Object Literals
var obj={};

This is the simplest way of creating an object in JavaScript. Objects in JavaScript can have properties and methods. To include the properties into the above object, simply bind it using the dot operator.

```
obj.firstName="Ankur";
obj.lastName="Aggarwal";
obj.fullName=function()
{
  return obj.firstName+obj.lastName
}
obj.fullName(); // will print Ankur Aggarwal
```

```
//new keyword
function info()
{
  this.firstName="Ankur";
  this.lastName="Aggarwal";
  this.fullName=function()
  {
    return obj.firstName+obj.lastName
  }
}
var obj=new info();
```

Another way of creating the object is by using a new